Week 6: Statistical Learning and Tree Predictors*

Instructor: Dr. Federico Fusco

November 4, 2025 - version 1

1 Generalization Bounds

Our goal is to provide a mathematical framework for the overfitting phenomenon. From previous weeks, we have the high-level intuition that "more complex" models require more data to achieve the same performance on the test data. This perspective is provided by the so-called **Generalization Bounds**, which quantify the number of samples required to ensure that the expected risk of *all the classifiers* is well estimated by the corresponding empirical risk. In these notes, we focus on binary classification, i.e., the label set $\mathcal{Y}$ is only composed of two labels ($\{0,1\}$ or $\{-1,1\}$, depending on the context), but similar results can also be achieved for multi-class classification and regression.

Assume that we have access to n i.i.d. samples from a distribution $\mathcal{D}$ over $\mathcal{X} \times \mathcal{Y}$, and we are focusing on a specific family $\mathcal{H}$ of classifiers. For each $h \in \mathcal{H}$, we use the empirical risk $\hat{R}$ to estimate the expected risk R. Stated differently, we are using the training set (i.e., the n i.i.d. samples) to make predictions on unseen samples (i.e., the test set).

$$R(h) = \mathbb{E}\left[\ell\left(h(X), Y\right)\right], \quad \hat{R}(h) = \frac{1}{n}\sum_{i=1}^{n} \ell\left(h(x_i), y_i\right).$$

We would like to argue that for *all* classifiers $h \in \mathcal{H}$, it holds that $\hat{R}(h) \approx R(h)$.

1.1 Mathematical Preliminaries

We start by recalling some fundamental results from probability theory: union bound and Chernoff inequality.

Theorem 1.1 (Union Bound)**.** *Given events $A_1, \dots, A_k$, then $\mathbb{P}\left(A_1 \cup \dots \cup A_k\right) \leq \sum_i \mathbb{P}\left(A_i\right)$.*

Theorem 1.2 (Chernoff Bound)**.** *Let $Z_1, \dots, Z_n$ be n independent and identically distributed Bernoulli random variable, with probability of success μ. Denote with $\hat{Z}$ the average of the n random variables, and denote with ε a precision parameter. We have the following:*

$$\mathbb{P}\left(|\hat{Z} - \mu| > \varepsilon\right) \leq 2e^{-2\varepsilon^2 n}$$

*These notes are meant as a teaching support for the students of the "Machine Learning" course for the Master's Degree in Engineering in Computer Science and Artificial Intelligence that I am teaching in Fall 2025. They are likely full of typos and imprecisions, so please get in touch with me in class if something feels off. From time to time, I may update new versions of these notes, so make sure to download the latest version.

1.2 Finite Families of Classifiers

When the family of classifiers we are considering is finite, then we have the following simple result.

Theorem 1.3. *Let $\varepsilon \in (0,1)$ be a fixed precision parameter. Consider a* finite *family of classifiers $\mathcal{H}$, if we have n i.i.d. samples, we get the following:*

$$\mathbb{P}\left(\max_{h \in H} |R(h) - \hat{R}(h)| > \varepsilon\right) \leq 2|\mathcal{H}|e^{-2\varepsilon^2 n}$$

Proof. Denote with $\mathcal{E}_h$ the clean event for the generic classifier $h \in \mathcal{H}$ as follows:

$$\mathcal{E}_h = \left\{|R(h) - \hat{R}(h)| \leq \varepsilon\right\}.$$

The event that we are trying to control, namely that for any $h \in \mathcal{H}$ the risk is well approximated corresponds to the intersection over all h of the clean events. Namely, we have the following:

$$\begin{aligned}
\mathbb{P}\left(\max_{h \in H} |R(h) - \hat{R}(h)| > \varepsilon\right) &= \mathbb{P}\left(\bigcup_{h \in \mathcal{H}} \mathcal{E}_h^C\right) \\
&\leq \sum_{h \in \mathcal{H}} \mathbb{P}\left(\mathcal{E}_h^C\right) && \text{(By Union Bound, see Theorem 1.1)} \\
&\leq 2|\mathcal{H}|e^{-2\varepsilon^2 n} && \text{(By Chernoff Bound, see Theorem 1.2)}
\end{aligned}$$

□

The above inequality relates the precision parameter ε and the number of samples n with the failure probability. In particular, if we fix the precision ε and the tolerated probability of failure $\delta \in (0,1)$, we get the following, more explicit result.

Corollary 1.4. *If $n \geq \frac{1}{2\varepsilon^2} \ln\left(\frac{2|\mathcal{H}|}{\delta}\right)$, then the probability of error is at most δ.*

> **Important**
>
> It is crucial to understand the randomness involved in the statement of the theorem: we consider the n i.i.d. samples, and we argue that, with probability at least $1 - \delta$ with respect to these samples, the expected performance on a *new* sample is ε-approximated by the empirical performance on the n initial samples.

1.3 Infinite Families of Classifiers: The VC Dimension

Although direct, the result for finite families is fairly unsatisfactory, as meaningful families of classifiers are typically infinite (e.g., linear classifiers). Luckily, there is a simple combinatorial notion that characterizes the families for which something similar to Theorem 1.3 can be obtained.

Definition 1.5 (Shattering)**.** *Given a set S of points $\{x_1, \ldots, x_{|S|}\}$, we say that family $\mathcal{H}$ shatters S, if $\mathcal{H}$ can realize any labeling of S: for any set of labels $\{y_1, \ldots, y_{|S|}\} \in \{0,1\}^{|S|}$, there exists $h \in \mathcal{H}$ such that $h(x_i) = y_i$ for all $x_i \in S$.*

Definition 1.6 (VC Dimension)**.** *Given a hypothesis class $\mathcal{H}$, we define its Vapnik-Chervonenkis (VC) dimension as the size of the largest set that is shattered by $\mathcal{H}$. If $\mathcal{H}$ can shatter arbitrarily large sets, then the VC dimension is set to infinity.*

We then have the following result, which is arguably the most important theorem in learning theory. The proof is omitted, but can be found in Chapter 14.5 of Mitzenmacher and Upfal [2017] or Chapter 6 of Shalev-Shwartz and Ben-David [2014].

Theorem 1.7. *Let $\varepsilon \in (0,1)$ be a fixed precision parameter, and $\delta \in (0,1)$ be the tolerated probability of error. Consider a family of classifiers $\mathcal{H}$ with VC dimension d, if we have n i.i.d. samples, with $n \in \Omega\left(\frac{d \log 1/\varepsilon + \log 1/\delta}{\varepsilon^2}\right)$ we get the following:*

$$\mathbb{P}\left(\sup_{h \in H} |R(h) - \hat{R}(h)| > \varepsilon\right) \leq \delta.$$

Note

The way of looking at the Theorem is as follows: given a precision ε and a tolerated probability of error δ, it tells how many samples are needed to ε-approximate the risk of all classifiers in $\mathcal{H}$. In particular, this number scales linearly with the VC dimension.

Example 1.8. *The VC dimension of intervals is* 2.

Example 1.9. *The VC dimension of axis-aligned rectangles in $\mathbb{R}^2$ is* 4.

Example 1.10. *The VC dimension of triangles in $\mathbb{R}^2$ is* 7

Example 1.11. *The VC dimension of a finite family of classifiers is $\log(\mathcal{H})$. We therefore have that Theorem 1.3 is a special case of Theorem 1.7.*

Important

It is surprising that a combinatorial quantity such as the VC dimension characterizes a probabilistic property. Note, the VC dimension does characterizes learnability; indeed, for any family with unbounded VC dimension, we cannot guarantee the same bounds as in Theorem 1.3 or Theorem 1.7: there will always be some classifiers whose risk is not well estimated!

1.4 Risk Decomposition

We can now provide a mathematically satisfactory exposition of overfitting, at least for the special case of binary classification. Let $h^\star$ denote the Bayes optimal classifier, and let $\mathcal{H}$ be any fixed family of classifiers. We can decompose the expected risk of a generic classifier $h \in \mathcal{H}$ as follows:

$$R(h) = \underbrace{R(h^\star)}_{\text{Bayes Error}} + \underbrace{\left(\inf_{h \in \mathcal{H}} R(h) - R(h^\star)\right)}_{\text{Bias given by class } \mathcal{H}} + \left(R(h) - \inf_{h \in \mathcal{H}} R(h)\right).$$

We analyze separately the three terms:

- The first term of this decomposition is the Bayes Error, i.e., the risk of the best classifier. This error is a priori non-zero, and there is nothing we can do to reduce it.
- The second term is the "discretization" error that we incur by restricting our attention to the family $\mathcal{H}$. It only depends on the choice of the family $\mathcal{H}$, and is not affected directly by the number of samples, nor by how we pick the classifier h.

- The third error is the risk of h with respect to the best classifier in $\mathcal{H}$. We know that for all classifiers in $\mathcal{H}$, the empirical risk $\hat{R}$ is a good estimate of R, up to an error that is quantified in *Theorem* 1.7. Denote with $h^\star_{\mathcal{H}}$ the best classifier in $\mathcal{H}$, we get

$$R(h) - R(h^\star_{\mathcal{H}}) \leq \hat{R}(h) - \hat{R}(h^\star_{\mathcal{H}}) + O\left(\sqrt{\frac{d \log n/d + \log 1/\delta}{n}}\right)$$

  In particular, if we let h to be the *best* classifiers on the samples (or nearly the best), then the above term is essentially $\sqrt{\frac{d \log n/d + \log 1/\delta}{n}}$, which only depends on the VC dimension d, the number of samples n, and the tolerated probability of error δ.

Important

The trade-off when choosing the "right" family $\mathcal{H}$ of classifiers is clear: the more complex the class, i.e., the larger the VC dimension, the smaller the second term becomes, at the cost of a worse bound on the third term. Similarly, choosing a simple family $\mathcal{H}$ makes it easier to control the third term, while incurring a large discretization error.

2 Tree Predictors

Assume that the input space $\mathcal{X}$ is given by the product of $\mathcal{X}_1 \times \ldots \mathcal{X}_d$, we consider the family of classifiers that can be represented by trees. A tree predictor is associated with a rooted tree whose internal nodes correspond to tests, while the leaves correspond to labels. To compute the label of $x \in \mathcal{X}$, we need to follow a root-to-leaves path, starting from the root, and following the outcomes of the tests. Typically, the tests only entail checking one dimension of x at a time.

Example 2.1. *Consider $\mathcal{X} = [0,1]^2$ and the predictor tree that works as follows: if $x^{(1)} \geq 1/2$, then move left, otherwise go right. If right, then output label 1 if $x^{(2)} \geq 1/3$, and -1 otherwise. If left then output label 1 if $x^{(2)} \leq 2/3$, and -1 otherwise.*

In these notes, we focus on binary classification (the label set $\mathcal{Y}$ is $\{-1, 1\}$), and binary input $\mathcal{X} = \{0,1\}^d$, and the 0-1 loss. The argument can be easily extended beyond the binary case and to more general losses. We also restrict our attention to binary tests (i.e., each internal node in the tree has exactly two children, depending on the binary outcome of the test).

2.1 Training a Tree Predictor

Consider a generic tree predictor h_T, we want to evaluate its performance on the training set $\{(x_1, y_1), \ldots (x_n, y_n)\}$. Following the structure of the tree predictor h_T, we can associate each one of the training points to a leaf ℓ. Since our goal is to minimize the number of points that are misclassified, it makes sense to label a leaf with the label that is more frequent there. Denote with N^+_ℓ, respectively N^-_ℓ, the number of points in the training set that gets mapped to leaf ℓ and have label $+1$, respectively -1, then we label ℓ with $+1$ if $N^+_\ell \geq N^-_\ell$, and with -1 otherwise. In particular, the number of errors in a given leaf is equal to the cardinality of the smallest class: $\min\left\{N^+_\ell, N^-_\ell\right\}$. All in all, the error of a tree predictor h_T can be written as:

$$\hat{R}(h_T) = \frac{1}{n} \sum_{\ell \text{ leaf}} \min\left\{\frac{N^+_\ell}{N_\ell}, \frac{N^-_\ell}{N_\ell}\right\} N_\ell = \frac{1}{n} \sum_{\ell \text{ leaf}} \psi\left(\frac{N^+_\ell}{N_\ell}\right) N_\ell,$$

where N_ℓ denotes the number of points in leave ℓ, and the function $\psi(z)$ is simply $\min\{z, 1-z\}$. Tree predictors exhibit a crucial difference with respect to all the other machine learning models that we have studied: if we let the tree to grow arbitrarily deep, then we can typically set the error to 0!

Theorem 2.2. *For any training set with distinct inputs, i.e., $x_i \neq x_j$ for any $i \neq j$, there exists a tree predictor that perfectly classifies all the points.*

Proof Sketch. We can achieve this result by simply constructing a tree where each input point is contained in exactly one leaf, and then correctly labeling that leaf. This can be achieved in many ways; for instance, it is possible to "split" the dataset one coordinate at a time. □

As you can imagine, such an approach wildly overfits on the training set, so we need an alternative perspective. The idea is to consider a good tree predictor that has at most N nodes, where N is the hyper-parameter that captures the complexity of the tree. Once we fix N, the goal becomes finding the best tree classifier with at most N nodes on the given training set. While this problem can be solved optimally, we present here a well-known greedy algorithm that exhibits nice empirical performance. It works as follows: starting from a trivial tree with only one leaf, it recursively chooses a leaf, replaces it with a test node, and connects the two corresponding leaves. Terminating as soon as the tree has N nodes. The algorithm works as follows:

Training a Decision Tree

Input: Training data $\{(x_i, y_i)\}_{i=1}^n$, where $x_i \in \mathbb{R}^d$, $y_i \in \{-1, +1\}$
Hyperparameter: Maximum number of nodes N
Start with a single leaf that contains all the points
while The decision tree has less than N nodes **do**
  Select a leaf ℓ that contains some misclassified points
  Replace it with a test node, and two leaves ℓ_L and ℓ_R
  Move the points in ℓ to the two children, according to the outcome of the test

Note

The greedy algorithm that we have described is *not* optimal, but performs reasonably well in practice.

Our goal is to find, given the training set, a tree predictor with "small" error. To this end, we make the following observation: if we start from a leaf and replace it with a test, then the training error cannot increase. Formally, if we split a leaf ℓ in two leaves ℓ_L and ℓ_R, we get:

$$\underbrace{\psi\left(\frac{N_\ell^+}{N_\ell}\right) N_\ell}_{\text{contribution of } \ell} = \psi\left(\frac{N_{\ell_L}^+}{N_{\ell_L}}\frac{N_{\ell_L}}{N_\ell} + \frac{N_{\ell_R}^+}{N_{\ell_R}}\frac{N_{\ell_R}}{N_\ell}\right) N_\ell \geq \underbrace{\psi\left(\frac{N_{\ell_L}^+}{N_{\ell_L}}\right) N_{\ell_L}}_{\text{contribution of } \ell_L} + \underbrace{\psi\left(\frac{N_{\ell_R}^+}{N_{\ell_R}}\right), N_{\ell_R}}_{\text{contribution of } \ell_R}$$

where in the inequality, we use that the function ψ is concave.

Remark 2.3. *The function $\psi(z) = \min\{z, 1-z\}$ may be problematic, as it could result in a non-decrease in the loss. Consider the instance where $\frac{N_\ell^+}{N_\ell} = 0.8$, $\frac{N_{\ell_L}^+}{N_{\ell_L}} = 0.6$, $\frac{N_{\ell_R}^+}{N_{\ell_R}} = 1$, and $\frac{N_{\ell_L}}{N_\ell} = 2$.*

Therefore, there are other choice of function ψ as "surrogate" of $\min\{z, 1-z\}$, and are used to choose which leaf to split and how to perform the split:

- Gini index $\psi_G(z) = 2z(1-z)$
- Scaled Entropy $\psi_e(z) = -\frac{z}{2}\log z - \frac{1-z}{2}\log 1 - z$

Important

A fantastic feature of trees is that they are easy to interpret.

2.2 The VC dimension of tree predictors

We can use the notion of VC dimension to formalize our intuition about the complexity of tree predictors. In particular, let $\mathcal{T}$ be the family of all tree predictors; it is a simple exercise to argue that it has infinite VC dimension. Indeed, given any disjoint n points, we can always find a tree deep enough that maps each point to a leaf, and thus can shatter the n points. On the other hand, the tree classifiers on N nodes $\mathcal{T}_N$ are finitely many, so it is possible to learn all such classifiers with enough samples (by Theorem 1.3).

Proposition 2.4. *Consider the family $\mathcal{T}_N$ of all the tree predictors on trees of N nodes, then $|\mathcal{T}_N| \leq (2de)^N$*

Proof. The cardinality of $\mathcal{T}_N$ is smaller than the product of: the number of binary trees with N nodes (this specifies the topology of the underlying tree), the number of ways of assigning binary tests to the internal nodes, and the number of ways of assigning binary labels to the leaves. If the tree has M internal nodes, then there are d^M ways of assigning tests to internal nodes (there are d dimensions), and 2^{N-M} ways of assigning the labels to the leaves. All in all, each tree of N nodes can implement up to $2^{N-M}d^M \leq d^N$ classifiers. Finally, the number of complete binary trees is given by the following formula:

$$\frac{2}{N+1}\binom{N-1}{\frac{N-1}{2}} \leq \frac{2}{N-1}(2e)^{\frac{N-1}{2}},$$

where in the second inequality we use the Stirling approximation, which implies that $\binom{n}{k} \leq (2n/k)^k$. All in all, we get: $|\mathcal{T}_N| \leq (2ed)^N$. □

The Proposition tells us that the number of samples needed to get good estimates scales linearly in N, and only logarithmically in the dimension of the input space d.

2.3 Random Forests

There is a powerful way of combining tree predictors into a unique *better* predictor, called Random Forests. Such an approach consists in instantiating multiple short tree predictors, trained on different subsets of the same training set. Each tree is associated with a weight, and the final prediction is determined by the weighted majority of the predictions from the individual trees.

3 Pointers

The main reference for generalization bounds and VC dimension is Shalev-Shwartz and Ben-David [2014]. We also refer to Mitzenmacher and Upfal [2017] for a more probabilistic approach, and to Ng [2022] for a more practical one. Regarding Tree Predictors and Random Forests, we refer to Chapter 6 of Géron [2019] for a more practical approach, and to Chapter 18 of Shalev-Shwartz and Ben-David [2014] and to the notes by Cesa-Bianchi [2025] for a more theoretical perspective.

References

Nicolò Cesa-Bianchi. Statistical methods for machine learning, 2025. Lecture notes available at `https://cesa-bianchi.di.unimi.it/MSA/index_25-26.html`.

Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media, Inc, second edition edition, 2019.

Michael Mitzenmacher and Eli Upfal. *Probability and Computing - Second Edition*. Cambridge University Press, 2017.

Andrew Ng. Cs229 lecture notes. Online course notes, 2022. Available at `https://cs229.stanford.edu/notes2022fall/main_notes.pdf`.

Shai Shalev-Shwartz and Shai Ben-David. *Understanding Machine Learning - From Theory to Algorithms*. Cambridge University Press, 2014. URL `https://dn721802.ca.archive.org/0/items/understanding-machine-learning-theory-algorithms_202404/understanding-machine-learning-theory-algorithms.pdf`.